17 min read

Foundations of Reinforcement Learning

RL Part 1: Agents, environments, rewards, and why RL is different from supervised learning.

Introduction

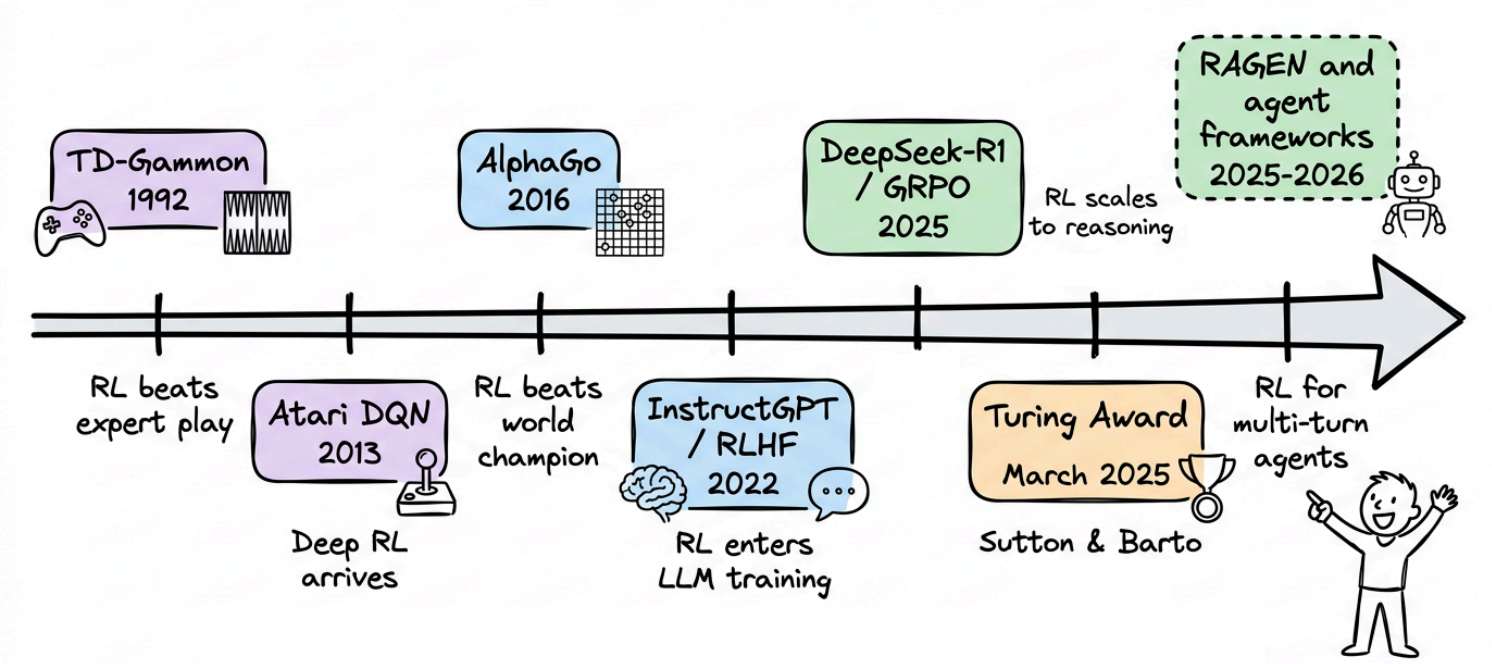
On 5 March 2025, the Association for Computing Machinery announced that Andrew G. Barto and Richard S. Sutton had won the 2024 ACM A.M. Turing Award, the computing world's equivalent of the Nobel Prize.

The citation was specific: "for developing the conceptual and algorithmic foundations of reinforcement learning." Their 1998 textbook, "Reinforcement Learning: An Introduction", has been cited over 75,000 times and is still the standard reference for the field.

But Reinforcement learning (RL) sat quietly for decades, producing impressive but niche results such as TD-Gammon in the 1990s and AlphaGo in 2016.

Then something shifted, almost every LLM released in the past two years (e.g. DeepSeek-R1, GPT-5, etc.) was trained using some form of reinforcement learning in its post-training pipeline.

RL, today, is a core part of how the most capable AI systems are built.



We're starting this RL series as a structured learning path, designed to build a thorough understanding while developing a strong conceptual intuition of the key topics and ideas.

Just as the MLOps and LLMOps crash course, each chapter will clearly explain necessary concepts, provide examples, diagrams, and implementations.

> 👉 Prerequisites: We assume comfort with Python, basic probability and linear algebra (expectations, simple distributions, vectors and matrices), and fundamental ML/DL concepts. No prior background in reinforcement learning is required. Some concepts will feel dense at times (they usually are), but don't worry, we will build them up slowly.

In this chapter, we'll understand what makes RL different from supervised and unsupervised learning, how to describe an agent interacting with an environment, why exploration is fundamentally unavoidable, and how all of this plays out in the simplest possible RL setting: the multi-armed bandit.

As always, every notion will be explained through clear examples and walkthroughs to develop a solid understanding.

Let's begin!

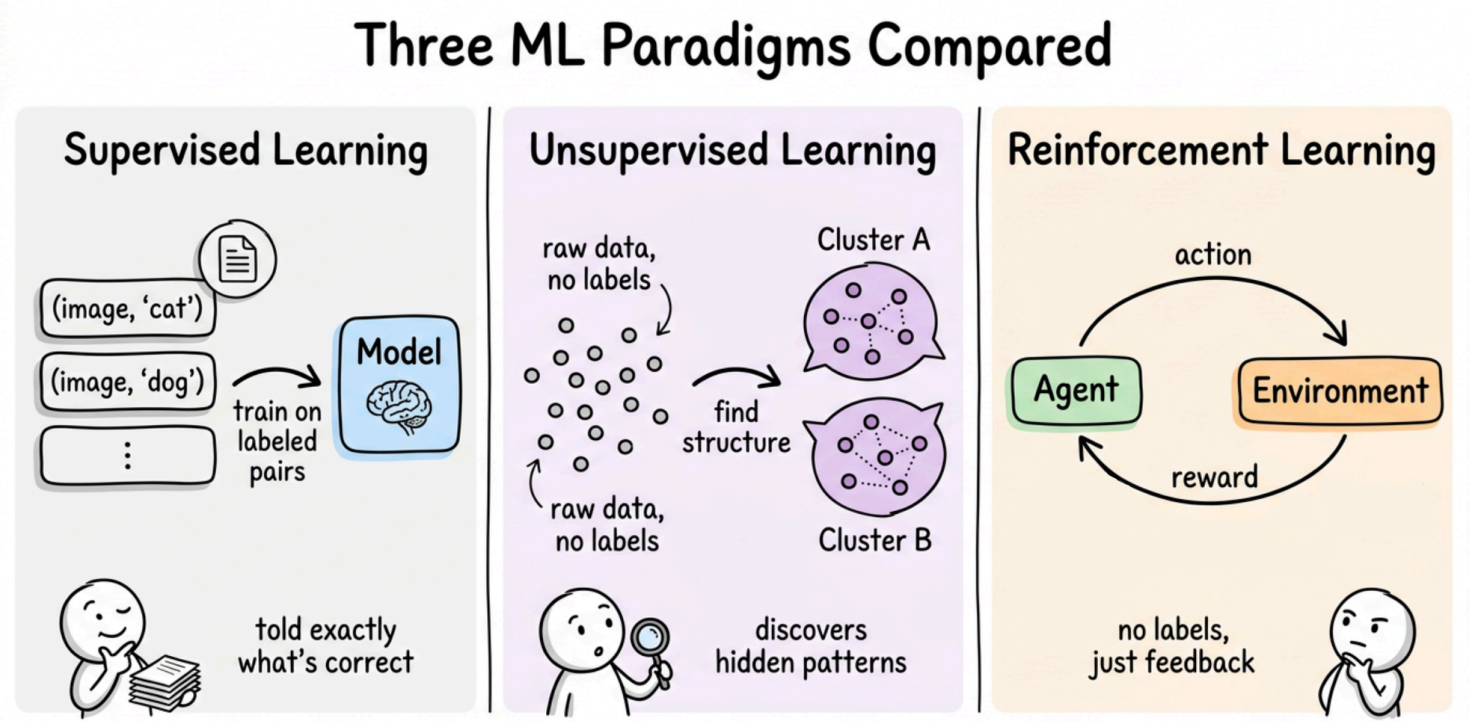
What makes RL different

Most machine learning that we have seen so far falls into two buckets:

- Supervised learning gives the model input-output pairs and asks it to learn the mapping.
- Unsupervised learning gives the model only inputs and asks it to find structure, like clusters in a dataset or a lower-dimensional representation.

Reinforcement learning is a third kind of machine learning, and it is a genuinely different setup.

There are no labeled pairs. There is no hidden structure to discover. Instead, there is an agent that takes actions in an environment, and after each action, the environment returns something called a reward.makeThe agent's goal is to pick actions so that the total reward it collects over time is as large as possible.



Properties that set RL apart

There are four key properties that make RL distinctive:

- The first is that feedback is evaluative, not instructive. A supervised loss function tells the model exactly what the correct output was. An RL reward just tells the agent how good its action was, not what the best action would have been. The agent has to figure out "best" on its own.

- The second is that the data distribution depends on the agent. In supervised learning, the training set is fixed. In RL, the states the agent ends up in are a consequence of its own choices. A bad early policy can steer the agent into regions of the environment it would never have visited with a good policy. This means the data is not independent and identically distributed (i.i.d.), and thus most guarantees from supervised learning do not carry over directly.

👉 In basic terms, a policy in reinforcement learning is the agent's strategy, rulebook, or mapping function that dictates which action to take in a given state to maximize cumulative rewards.

- The third is delayed consequences. An action taken now might only pay off many steps later.

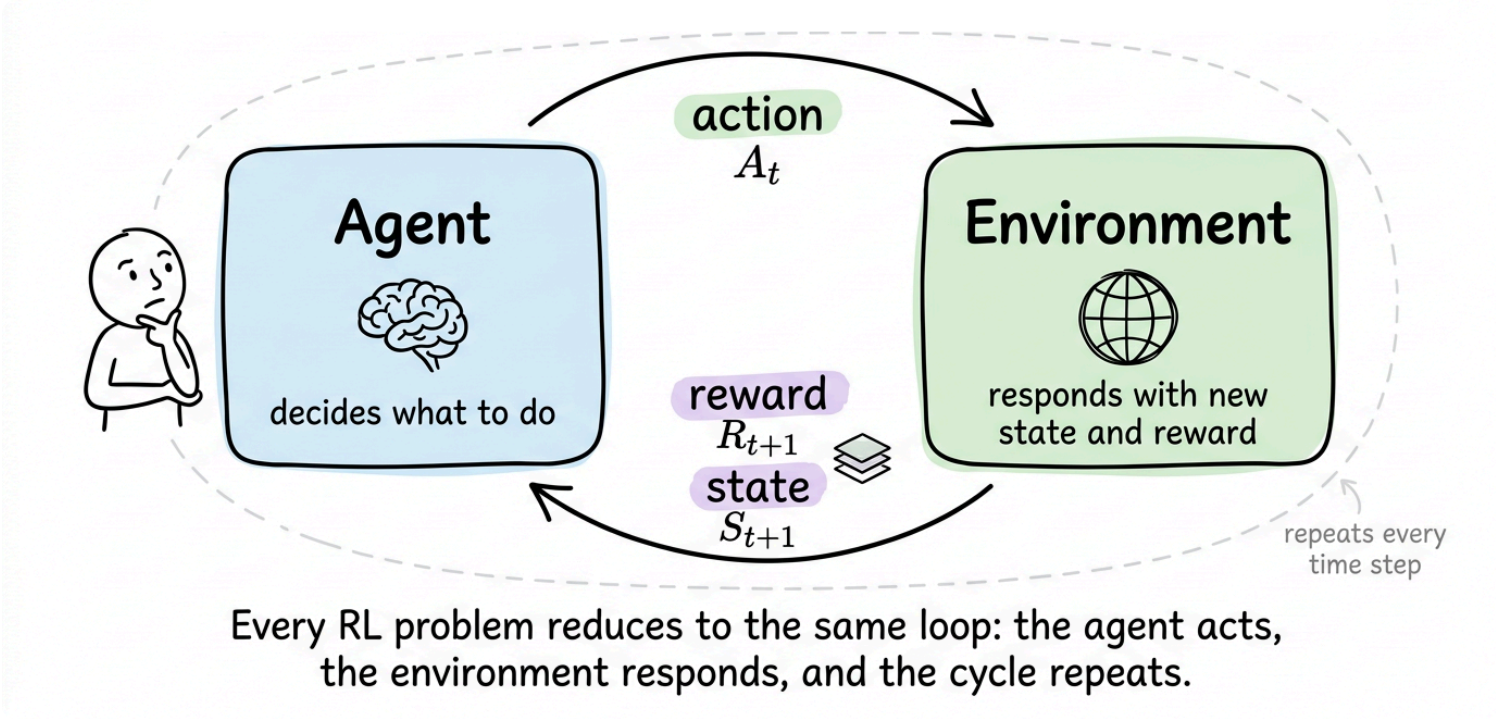
👉 Figuring out which of many earlier actions was actually responsible for a later reward is called the credit assignment problem, and it is one of RL's central difficulties.

- The fourth is the exploration-exploitation tradeoff. The agent has to use what it knows to pick good actions. But it also has to try actions it is uncertain about, in case they turn out to be even better. This tension does not exist in supervised learning, where the labels are given.

> 👉 RL is sometimes called "learning from interaction" or "closed-loop learning". The loop is important, the agent's output influences its future input. In supervised learning, the training data is a closed file that will not change no matter what the model does. In RL, the model's own behaviour shapes what it sees next.

The agent-environment loop

The canonical RL diagram shows two boxes, one labeled "agent" and one labeled "environment", with arrows between them. Every RL problem that we will study in this series fits inside this picture.



The interaction

Time proceeds in discrete steps $t = 0, 1, 2, \ldots$. At each step, three things happen in order:

- The agent observes the current state of the environment, written S_t.

> 👉 A state is a description of the situation the agent is in, enough to decide what to do next. In a grid, for example, the state might be the agent's

coordinates. In chess, the state is the board position. In a dialogue model, the state could be the conversation history so far.

- The agent picks an action A_t based on what it sees.

👉 The action is the agent's output, the only way it can influence the environment. In chess, for example, an action is a legal move.

- The environment then does two things. It transitions to a new state S_{t+1}, and it emits a reward R_{t+1}, a scalar number that evaluates the previous action. The next time step begins and the loop continues.

Stringing this together gives a trajectory, also called an episode when it has a clear end:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots$$

Reading left to right, this is the entire history of the agent's interaction. Each $(S_t, A_t, R_{t+1}, S_{t+1})$ quartet is one transition, and much of RL is about learning from such transitions.

Where does the agent end and the environment begin?

This question might sound philosophical but it has a concrete answer that Sutton and Barto crystallized. The boundary between agent and environment is drawn wherever the agent's direct control ends.

Anything the agent cannot change arbitrarily and instantly is part of the environment, even if it is physically inside the agent's body.

> 👉 A point to note is, this is a modeling choice, not a physical fact. You draw the boundary where it is useful for the problem you are trying to solve. Drawing it in different places changes what counts as a state, what counts as an action, and what the environment's dynamics look like.

The tradeoff here is practical:

- Drawing the boundary tightly (almost everything is environment) keeps the action space small and clean but leaves the agent with less control.
- Drawing it loosely (a lot of internal state is "agent") gives the agent more levers but makes learning harder because the action space explodes.

With the interaction picture in place, we now need to understand how does the agent figure out the right policy when it starts out knowing nothing?

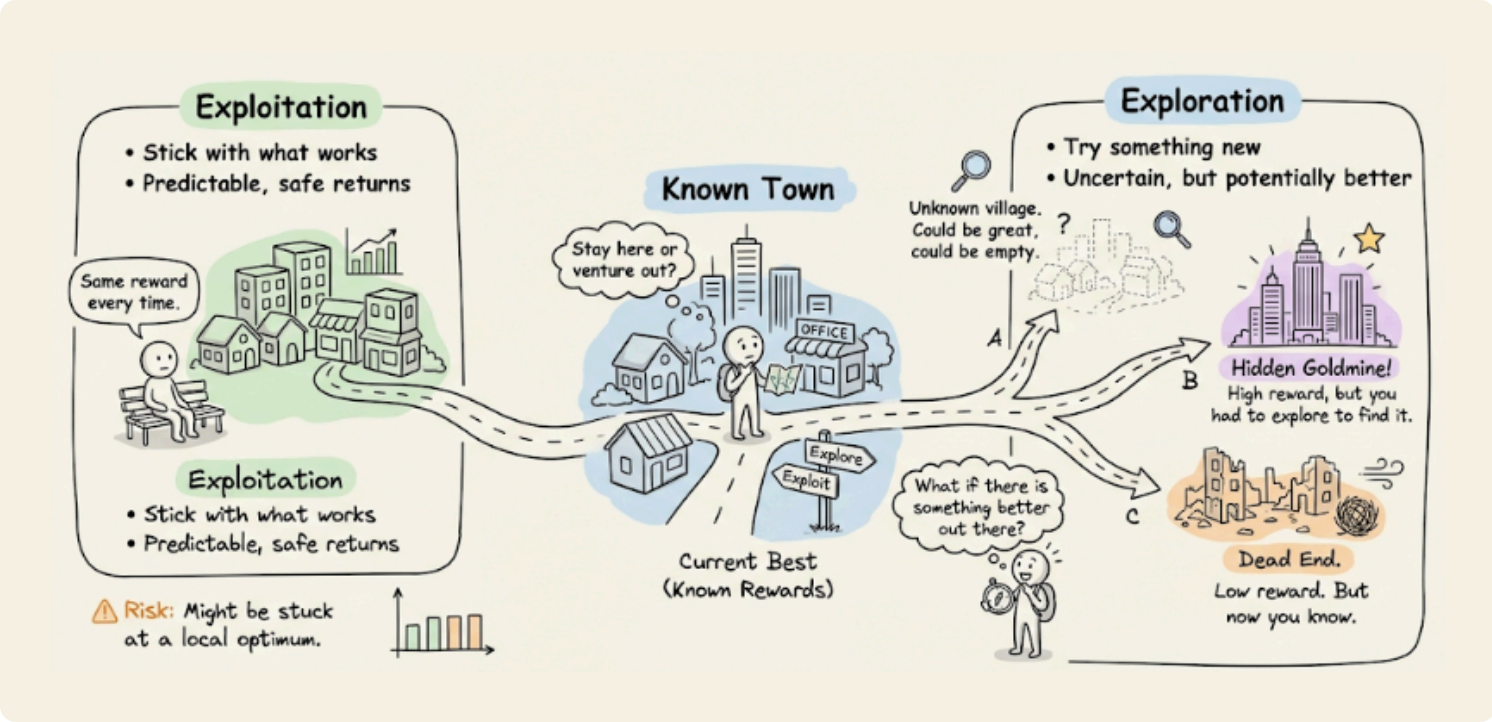
Exploration and exploitation

Suppose the agent has been interacting with an environment for a while and has some sense of which actions tend to produce good rewards. At each step, it faces a choice:

Should it take the action that looks best based on what it has seen so far?

Or

Should it try something different?



This is the exploration-exploitation tradeoff, and it is the defining tension of RL.

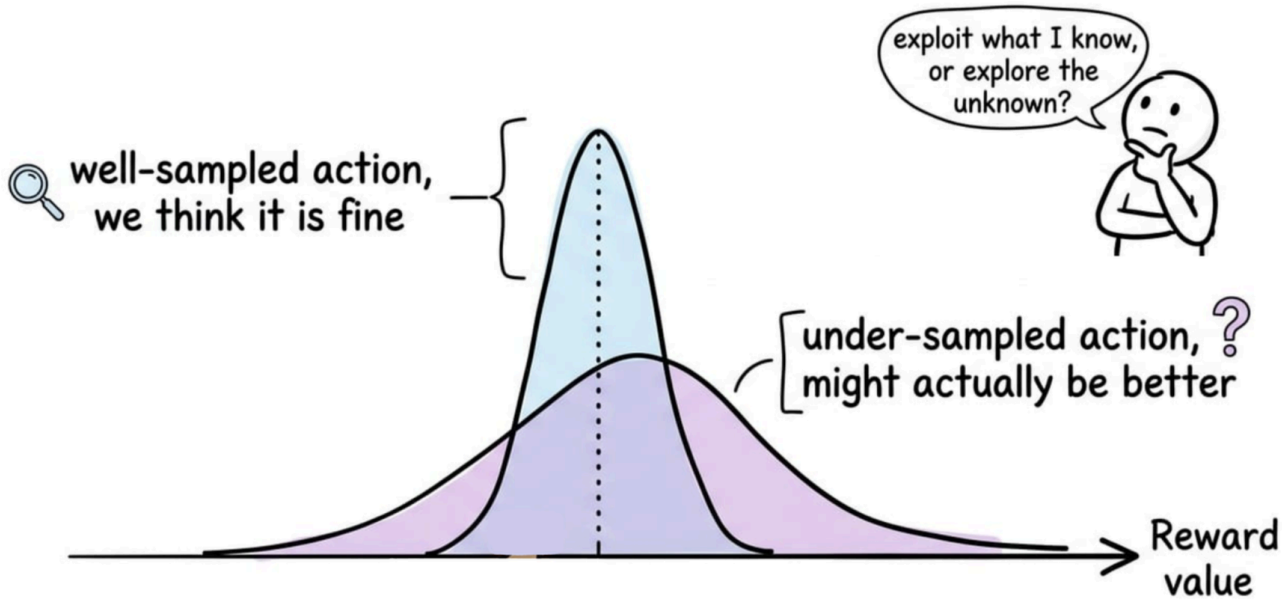
Exploitation means using current knowledge to pick the action that currently looks best. This is what maximizes immediate expected reward given what the agent knows.

Exploration means deliberately picking a different action to gather more information about it. This does not maximize immediate expected reward, but it can pay off later if the action turns out to be better than expected.

Pure exploitation fails. If the agent always picks the action that looks best after only a few tries, it will often lock in on a suboptimal choice it happened to sample well early on.

Pure exploration also fails. If the agent never uses what it has learned, it collects endless data but never turns it into reward.

One way to picture this is that for each action, the agent holds a belief about its reward (not a single number, but a distribution reflecting how sure it is).



- A well-sampled action has a narrow belief; the agent is confident about roughly where its reward lies.
- An under-sampled action has a wide belief; its true value could plausibly be much higher or much lower than the current estimate. Exploration is worthwhile precisely because of that upper tail.

> 👉 Even if the two actions look comparable on average, the uncertain one carries real upside, a chance it turns out to be genuinely better than the action we currently favor.

This tradeoff shows up in every RL problem. It is cleanest in a setting where there are no states to worry about, just actions and rewards. That setting is the multi-armed bandit, and studying it carefully is how we will build intuition for the rest of the series.

Multi-armed bandits

The multi-armed bandit problem is the simplest RL setting we can study. The name comes from casino slot machines, each of which is sometimes called a "one-armed bandit" because it has one lever and tends to take your money.

A multi-armed bandit is a row of k such machines, each with its own unknown payout distribution, and the player has to decide which levers to pull.

The stripped-down setup is:

- There are k actions (the "arms").
- Each arm a has a true expected reward $q_*(a)$, which is unknown to the agent.
- At each step, the agent picks an arm, pulls it, and receives a reward sampled from that arm's distribution.
- The agent wants to maximize total reward over some number of pulls, for example 1000.

In this problem, there is no state. Every pull is independent of the last, and what happens next does not depend on what the agent did before.

The problem is fundamentally about the exploration-exploitation tradeoff. If we can solve this, we have understood one of RL's hardest ideas in its purest form.

Estimating action values from data

The agent does not know $q_*(a)$, but it can estimate it from experience. The most natural estimate is the sample average of rewards received from that arm:

Or, we can say, if before time t, arm a has been pulled n times with rewards $R_1, R_2, \ldots, R_n$, then:

$$Q_t(a) = \frac{R_1 + R_2 + \cdots + R_n}{n}$$

By the law of large numbers, as we pull the arm more and more: $Q_t(a) \to q_*(a)$.

👉 Law of large numbers states that the average of the results obtained from a large number of independent random samples converges to the true value.

Computing this average naively would be wasteful. There is an incremental update that maintains the same estimate using only the previous estimate and the new reward. Letting n count how many times a specific arm has been pulled so far:

$$Q_{n+1} = Q_n + \frac{1}{n}[R_n - Q_n]$$

Unpacking:

- Q_n is the estimate going into the n-th pull, formed from $n-1$ pulls.
- R_n is the reward from the n-th pull.
- $R_n - Q_n$ is the prediction error, i.e., how much the new reward differs from what we expected.
- $\frac{1}{n}$ is the step size, shrinking over time as we collect more samples. Thus, later samples nudge the estimate less.

Intuitively, the structure of this update is:

With the sample-average estimator and its incremental update in hand, we now have everything the agent needs to evaluate arms. What's still missing is a rule for choosing between them.

At each step, the agent has to commit to one arm, and its current Q_n values are the only evidence it has. Should it pick the arm that looks best right now, or hedge toward arms it's less sure about?

This is where the exploration-exploitation tradeoff becomes concrete in practice. Let’s now walk through four strategies, moving from the most naive to the more principled approaches.

Strategy 1: greedy

The greedy strategy always picks the action with the highest current estimate:

This is pure exploitation.

It sounds sensible but fails badly in practice. If the agent happens to get a lucky first pull on a mediocre arm, that arm's estimate becomes the highest and

greedy keeps pulling it forever. It never explores the other arms, so it never discovers the truly best one.

Strategy 2: ε-greedy

To improve on greedy strategy. the minimal fix is to mix in a bit of random exploration. With probability ε (typically a small number like 0.1), pick a random action. Otherwise, be greedy:

This is called ε-greedy, and despite its crudeness, it works surprisingly well.

With enough time, every arm gets pulled infinitely often (because of the random exploration), so every $Q_t(a)$ eventually converges to $q_*(a)$, plus the greedy part of the policy picks the best arm most of the time.

The tradeoff, however, is that ε-greedy keeps exploring forever at the same rate, even after it has figured out which arm is best. A common variation anneals ε toward zero over time, but then you have a new hyperparameter (the annealing schedule) to tune.

Strategy 3: optimistic initial values

A quieter way to encourage exploration is to start with $Q_0(a)$ set to a large positive number, higher than any realistic reward. A greedy policy on top of this will initially find that every arm's estimate is too high, and each pull will bring the estimate down toward the true value.

The agent will naturally try every arm a few times before settling. No explicit randomness needed.

> 👉 Optimistic initialization is elegant for stationary problems but does not help in nonstationary ones, where the true values drift over time and the optimism from step zero has long since worn off.

Strategy 4: upper confidence bound (UCB)

ε-greedy explores uniformly at random, wasting pulls on arms that are clearly bad. A smarter approach is to explore arms in proportion to how uncertain we are about them. The upper confidence bound (UCB) rule is:

Here:

- $Q_t(a)$ is the current value estimate for arm a.
- $N_t(a)$ is the number of times a has been pulled before time t.
- $\ln t$ is the natural logarithm of the current time step, which grows slowly.
- c is a constant controlling the exploration strength, typically around 2.
- The whole square-root term is the exploration bonus added to $Q_t(a)$.

Reading structurally, UCB picks the arm that maximizes "estimate + bonus". The bonus is large when $N_t(a)$ is small (i.e. we have not pulled this arm much) and

small when $N_t(a)$ is large (we already know this arm well). The $\ln t$ in the numerator ensures that as time passes, even well-sampled arms eventually get an exploration bump.

UCB usually outperforms ε-greedy on the classic testbed. The reason is that the bonus shrinks as an arm is pulled more, so UCB naturally moves on once an arm is well understood. This concentrates its exploration on arms where the agent is still genuinely uncertain. The result is that UCB learns which arm is best faster, and stops wasting pulls on arms it already knows are bad.

> 👉 UCB's guarantees rely on stationary rewards and an enumerable action set. Extending it to non-stationary settings requires modification.

Comparing strategies

Strategy	Idea	Parameters	Notes
Greedy	Always pick best current estimate	None	Locks in on lucky early wins.
ε-greedy	Random exploration with prob. ε	ε	Simple and robust, keeps exploring forever.
Optimistic init	Start estimates high	Initial value	Free early exploration, stationary only.

Strategy	Idea	Parameters	Notes
UCB	Exploration bonus	c	Usually better on standard testbed, assumes stationarit

These tradeoffs are easier to see than to describe. Next, in the hands-on section, we'll run each strategy on a fixed bandit and compare their learning curves.

Hands-on: the 10-armed testbed

Time to build the things we have been talking about. We will implement the classic 10-armed testbed, and use it to compare greedy, ε-greedy with two values of ε, and UCB.

By the end we will have plots showing exactly how each strategy learns over time, and concrete numbers for how well they do.

Setup

Download the zip file below:

bandits

bandits.zip • 232 KB

For details about the versions and dependencies, you can check the `.python-version` and `pyproject.toml` files.

👉 It is recommended to follow the explanation ahead side-by-side with the above attached project for a more comprehensive understanding.

The testbed setup

The 10-armed testbed works like this:

- We have $k = 10$ arms.
- The true mean reward for each arm, $q_*(a)$, is drawn once from a standard normal $\mathcal{N}(0, 1)$. Once drawn, these means stay fixed for the rest of the run (this is a stationary bandit).
- Each time the agent pulls arm a, the reward is sampled from $\mathcal{N}(q_*(a), 1)$.

To average out the randomness in any single run, we repeat the experiment on 2000 independently generated bandits, each for 1000 steps, and average the results. This produces smooth, interpretable curves.

Now let's look at the implementation:

`Bandit` class is the environment. Here we:

- Draw the k true mean rewards $q_*(a)$ once, from a standard normal $\mathcal{N}(0, 1)$, and store them in `q_star`. These values are then fixed for the lifetime of the object, which is what makes the bandit "stationary".

- Record `optimal_action`, the index of the best arm. The agent never sees this; it is kept for our own bookkeeping so we can later measure how often the agent picks the genuinely best arm.
- Define the `pull` method, which returns a reward sampled fresh from $\mathcal{N}(q_*(a), 1)$ every time it is called. Repeated pulls of the same arm give different numbers fluctuating around that arm's true mean, and this noise is exactly what makes the problem nontrivial. If pulls were deterministic, one sample would reveal each arm's value and there would be nothing to learn.
- The `seed` argument fixes the random number generator so a given seed always produces the same bandit task, which is what lets us average cleanly across 2,000 independent runs later on.

Next up we have our implementations of strategies:

Here:

- `EpsilonGreedy` class:
  - Maintains two NumPy arrays: `Q`, holding the current value estimate for each arm, and `N`, the count of how many times each arm has been pulled. Both start at zero.
  - The `select_action` method implements the ε-greedy rule: with probability ε, it samples a random arm (the exploration branch); otherwise, it picks the arm with the highest current estimate (the

exploitation branch). When multiple arms are tied at the top, it finds all of them and `rng.choice` picks one at random.

- The `update` method applies the incremental average rule we derived earlier. First it increments the pull count for the chosen arm. Then it nudges the estimate toward the observed reward, with step size $1/N$.

- `UCB` class:
  - Same bookkeeping as `EpsilonGreedy`: `Q` for estimates, `N` for counts, plus one extra piece: `self.t`, a global step counter incremented on every action selection. This counter is the t that appears inside $\ln t$ in the UCB formula.
  - The `select_action` method first checks whether any arm has never been pulled. If so, it picks one of those untried arms at random. This handles the edge case where `N[a] = 0` would technically be a division-by-zero. Forcing every arm to be tried at least once is the standard fix.
  - Once all arms have been tried, the agent computes the full UCB score `Q + c * sqrt(ln(t) / N)` as a vectorized NumPy expression, one number per arm, and picks the arm with the highest score (with random tie-breaking).
  - The `update` method is same as earlier: incremental average.

Now finally let's take a look at the code that helps us run our experiment:

Here:

- `run_experiment` function is the training loop that glues the bandit environment and the agent together.
    - It takes an `agent_factory` (a function that produces fresh agents on demand), the number of independent runs, and the number of steps per run.
    - Two result arrays are preallocated: `rewards[run, step]` to hold the reward received on each step of each run, and `optimal[run, step]` to hold a boolean flagging whether that step's action was the truly best arm.
    - The outer loop creates a fresh bandit and a fresh agent for each of the `n_runs` runs, seeding both for reproducibility. Crucially, each run gets a different bandit, so the 2,000 runs span 2,000 distinct bandit problems.
    - The inner loop runs the RL interaction cycle: the agent picks an action, the environment returns a reward, the agent updates its estimates, and the statistics are recorded.
    - After all runs finish, the function returns two 1D arrays of length `n_steps`:
        - the average reward at each step

- the percentage of optimal actions at each step, both averaged across the 2,000 independent runs.

> 👉 Averaging is what smooths out the noise from any single bandit task and gives the clean learning curves we plot.

- `main` function is the orchestrator.
  - It defines the four configurations to compare, using `lambda seed: ...` closures so each configuration can be instantiated as a fresh agent inside `run_experiment`.
  - The loop runs each configuration for 2,000 bandit tasks of 1,000 steps each, prints a final-step summary, and stores the full learning curves for plotting.

Results

Running the four configurations (greedy, $\varepsilon = 0.01$, $\varepsilon = 0.1$, UCB with $c = 2$) for 2000 runs of 1000 steps each, here is what happens at step 1000:

The ordering matches the theory:

- Greedy plateaus quickly because it stops exploring, and on average only finds the true best arm in a third of runs.
- $\varepsilon = 0.01$ is too cautious, it does find the best arm eventually but slowly.
- $\varepsilon = 0.1$ learns faster and ends up picking the best arm about 80 percent of the time.
- UCB beats them all, because it focuses its exploration on uncertain arms rather than exploring uniformly.

One subtle thing worth noticing in the plots: greedy's average reward rises very fast in the first few steps (it immediately exploits whatever looked good early),

but then stalls. ε-greedy learns slower but keeps improving. This shape is the signature of the exploration-exploitation tradeoff: the strategy that is best right now is rarely the one that will be best in a thousand steps.

Alright now, with this we complete the walkthrough of our demo and conclude the discussion for this chapter. In the upcoming chapters, we will continue to build on the core ideas of RL, and explore concepts and their implementations wherever applicable.

Conclusion

In this chapter, we got ourselves familiar with some ideas of reinforcement learning.

We saw what separates RL from supervised and unsupervised learning: evaluative feedback instead of labels, data distributions shaped by the agent's own actions, delayed consequences and the credit assignment problem, and the unavoidable exploration-exploitation tradeoff.

We then formalized the agent-environment loop as a trajectory of states, actions, and rewards, with the boundary between agent and environment drawn wherever the agent's direct control ends.

The multi-armed bandit gave us the cleanest setting to isolate exploration and exploitation. We looked at greedy, ε-greedy, optimistic initialization, and UCB.

Finally, we experimented on the 10-armed testbed. The results matched the theory: smarter exploration leads to better long-run behavior, and UCB edged out ε-greedy by focusing its exploration on uncertain arms.

In the next chapter, we will continue to build on the core ideas of RL like the Markov Decision Process (MDP), value functions, and more.

The aim, as always, is to build a strong conceptual foundation and equip you with a flexible framework for reasoning about the core principles and the broader landscape in general.

As always, thanks for reading!

Any questions?

Feel free to post them in the comments.

Or

If you wish to connect privately, feel free to initiate a chat here:

A daily colur
practices on
professional

Menu

Contact

FAQ

Published on Apr 26, 2026

1 Comments Share

Next

Markov Decision Processes and Value Functions >